



Data Foundations

AICB-P1 · Ngày 7 · Embedding & Vector Store

Trần Quang Thiện

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?”

Ví dụ: Agent CS dùng GPT-4 nhưng trả sai chính sách hoàn tiền — vì data là policy cũ 2023.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Data strategy cho AI product
2. Agent memory architecture
3. Embeddings
4. Vector store
5. Kết nối agent với data
6. Chunking & retrieval basics
7. Lab 7 + deliverable
8. Preview Day 08

- Hiểu **tầm quan trọng của dữ liệu** cho AI — data quality thường quyết định hơn model selection
- Phân biệt được **knowledge data, operational data, contextual data**
- Hiểu **embedding** là lớp biểu diễn nghĩa, không chỉ là một API call
- Hiểu được các bước và phương pháp **xử lý dữ liệu** trước khi đưa vào AI
- Hiểu **vector store** lưu gì, tìm gì, và lọc bằng metadata ra sao
- Mô tả được pipeline **Document → Chunk → Embed → Store → Query → Inject**
- Build được một **mini retrieval integration** để nối agent với dữ liệu riêng

`src/` (code hoàn chỉnh) + `report/REPORT.md` (1 báo cáo / sinh viên)

- `src/chunking.py` — 3 chunking strategies + cosine similarity
- `src/store.py` — EmbeddingStore với search, filter, delete
- `src/agent.py` — KnowledgeBaseAgent (RAG pattern)
- 5 benchmark queries + so sánh kết quả giữa các strategy trong nhóm

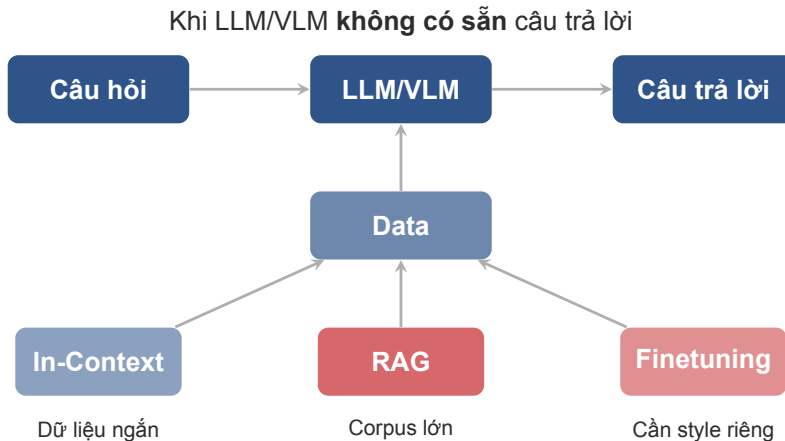
Data Strategy Cho Sản Phẩm AI

Không phải dữ liệu nào cũng nên đưa vào agent; chọn đúng dữ liệu quan trọng hơn nạp thật nhiều

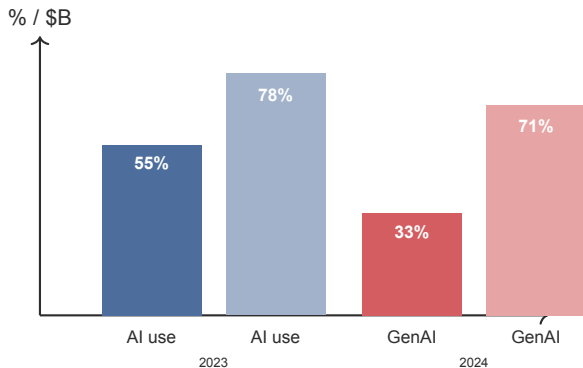
Khi LLM/VLM **biết** câu trả lời



Đơn giản — nhưng điều này hiếm khi đủ cho sản phẩm thật.



*Day 07 tập trung vào nhánh **RAG**: đưa đúng dữ liệu vào agent thay vì phải đổi model.*



- AI adoption đã tăng rất nhanh.
- Câu hỏi chuyển từ “dùng model nào” sang “agent được phép biết gì”.
- Nếu dữ liệu sai, cũ, bẩn, hoặc không truy được đúng lúc, output sẽ yếu dù model mạnh.

Based on Stanford AI Index 2025.

Lưu ý: Data quality và retrieval quality thường quyết định trải nghiệm thật hơn là đổi sang model đắt hơn.

Knowledge Data

Tài liệu, policy, SOP, FAQ, manual, hợp đồng, bài viết nội bộ.

Operational Data

Database, trạng thái đơn hàng, ticket, CRM records, logs, transactions.

Contextual Data

Session history, user profile, preferences, recent actions, channel context.

Knowledge data phù hợp với retrieval; **operational data** thường cần query có kiểm soát; **contextual data** nên được inject ngắn và đúng lúc.

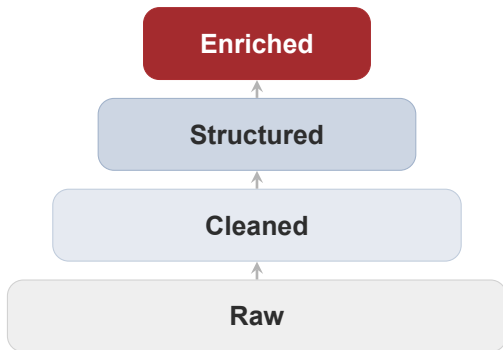
Nhóm bạn đang build một **AI assistant cho cửa hàng thời trang online**. Khách hàng chat để hỏi về sản phẩm, đơn hàng, và đổi trả.

Nhiệm vụ (10 phút):

1. Liệt kê ít nhất **2 ví dụ cụ thể** cho mỗi loại data:
Knowledge · **Operational** · **Contextual**
2. Với mỗi ví dụ, ghi rõ: data đó nên được **đưa vào agent bằng cách nào?**
(In-context? RAG? Query DB? Inject từ session?)
3. Viết ra giấy hoặc note chung trong nhóm.

Trình bày: 1 nhóm lên chia sẻ (2 phút), các nhóm khác bổ sung hoặc phản biện.

Gợi ý nếu bí: “Đơn hàng 12345 đang giao” thuộc loại nào? “Bảng size áo” thuộc loại nào? “User này hay mua size M” thuộc loại nào?



Tầng	Ví dụ cụ thể
Raw	PDF scan lộn, OCR ra “đổi trả”, HTML chứa tag rác
Cleaned	“đổi trả”, bỏ header/footer, chuẩn hóa Unicode
Structured	Chunk theo heading, gắn source refund-v3.pdf
Enriched	Tag category:support, access:public, quality:verified

Lưu ý: Sai lầm phổ biến: index ngay dữ liệu **raw** rồi kỳ vọng retrieval sẽ tự chữa mọi vấn đề.

- ☒ **Ai sở hữu dữ liệu?** team nào chịu trách nhiệm cập nhật?
- ☒ **Ai được phép truy cập?** dữ liệu nào cần ACL, dữ liệu nào public nội bộ?
- ☒ **Data freshness** bao lâu phải re-index?
- ☒ **PII / sensitive fields** có cần mask trước khi embed không?
- ☐ **Không index mọi thứ theo kiểu “cứ nạp hết vào vector DB đã”.**

Ví dụ: FAQ do team CS sở hữu, public nội bộ, re-index mỗi tuần, không chứa PII.

Agent Memory Architecture

Memory không chỉ là lưu nhiều hơn; nó là quyết định xem agent được nhớ gì, quên gì, và truy gì khi cần

Short-term Memory

- Nằm trong context window
- Giữ lịch sử gần đây và task hiện tại
- Rẻ để dùng, nhưng dễ đầy token
- Phù hợp cho session logic ngắn

Long-term Memory

- Nằm ngoài context window
- Thường là vector store, DB, profile store
- Phải truy xuất khi cần
- Phù hợp cho tri thức tích lũy và user history chọn lọc

Context window không phải là vector store. Agent chỉ “nhớ dài hạn” khi có cơ chế truy xuất hoặc lưu trữ bên ngoài.

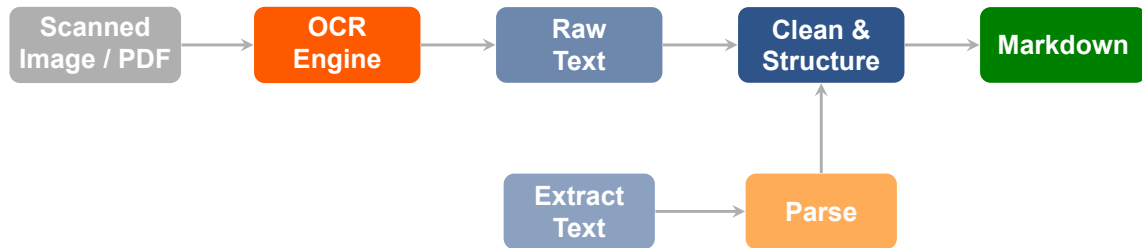
Data Processing

Xử lý dữ liệu để phục vụ cho AI rất quan trọng vì ảnh hưởng trực tiếp đến chất lượng câu trả lời

03

Format	Token hiệu quả	Giữ cấu trúc	Khi nào dùng
Markdown	★★★★★	Heading, list, table, bold	Mặc định được lựa chọn trong nhiều trường hợp
HTML (clean)	★★★	Bảng lồng, colspan, layout phức tạp	Khi MD mất thông tin cấu trúc
Plain Text	★★★★★	Không có cấu trúc	Email, chat log, transcript
JSON / YAML	★★★	Key-value, nested objects	Structured output, function calling

Nên chọn **Markdown**. So với HTML cùng nội dung, tiết kiệm ~30–50% token vì không có closing tags, attributes, class names. LLM được train trên lượng lớn MD nên hiểu rất tự nhiên.



*Đường thay thế cho tài liệu
có text sẵn (Word, HTML)*

Lưu ý: Sai lầm phổ biến: bỏ qua bước **Clean & Structure**, đưa raw text thẳng vào chunking. Kết quả: chunk chứa header/footer rác, OCR lỗi, ký tự đặc biệt — retrieval quality giảm rõ rệt.

Embeddings

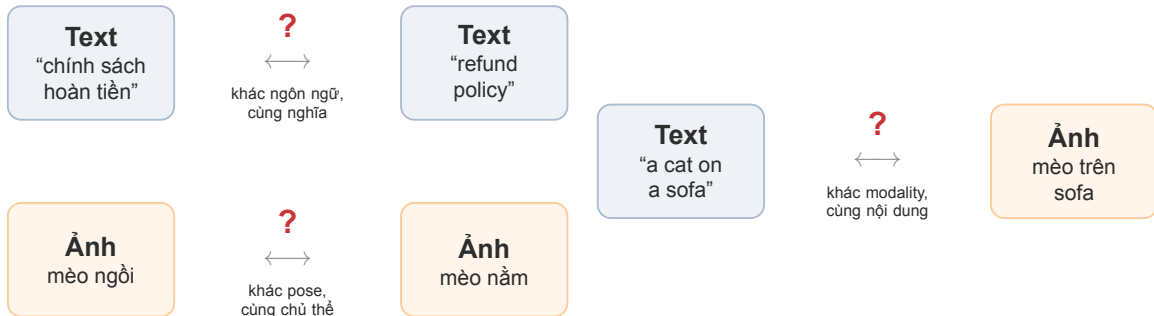
Embedding biến ngôn ngữ thành không gian toán học để máy có thể so sánh nghĩa thay vì chỉ so chữ

04

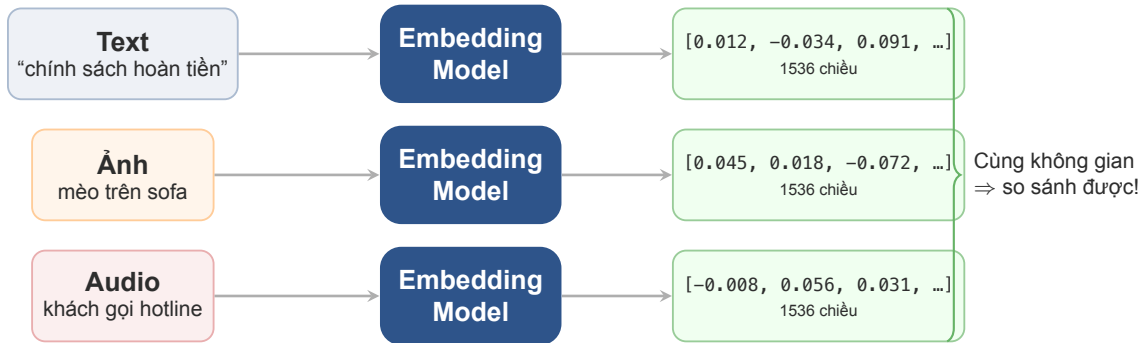
HÃY SUY NGHĨ...

“Làm sao để máy biết hai thứ “giống nhau” — khi chúng không cùng ngôn ngữ, không cùng định dạng?”

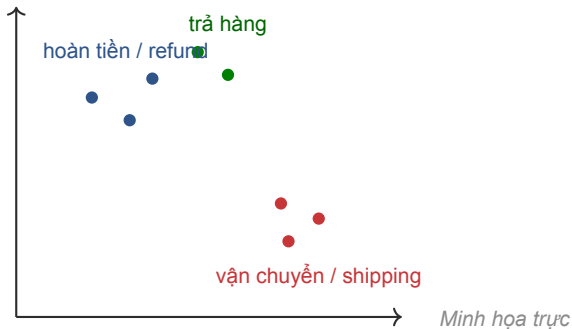
Giữ câu hỏi này trong đầu khi học bài hôm nay



Con người nhìn là biết “giống nhau”. Nhưng máy cần biến text, ảnh, audio thành **số** trong cùng không gian — rồi mới đo khoảng cách được.

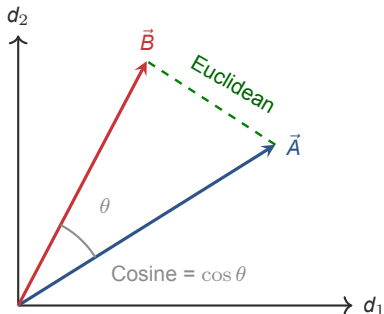


Embedding model là hàm biến dữ liệu thô (text, ảnh, audio) thành **vector số** cùng kích thước. Sau đó, đo khoảng cách giữa các vector = đo “độ giống nhau” về nghĩa.



quan: câu có nghĩa gần nhau sẽ ở gần nhau hơn trong vector space.

- Text được biến thành vector nhiều chiều.
- Từ đó, máy có thể đo **độ gần về nghĩa**.
- Đây là nền tảng cho semantic search, clustering, dedup, recommendation.



Cosine similarity

- Đo **góc** giữa hai vector
- -1 đến 1 ; càng gần 1 = càng giống nghĩa
- Không bị ảnh hưởng bởi norm
- **Dùng nhiều nhất** trong NLP và retrieval

Euclidean distance

- Đo **khoảng cách thẳng** giữa hai điểm
- Càng nhỏ = càng gần
- Bị ảnh hưởng bởi scale / norm

Cosine là mặc định cho text embedding (so nghĩa, bỏ qua độ dài). **Euclidean** phù hợp khi cần đo khoảng cách tuyệt đối (image, geo).

Cosine Similarity

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

- Tử: tích vô hướng (dot product)
- Mẫu: tích hai độ dài (chuẩn hóa)
- 1 = cùng hướng, 0 = vuông góc, -1 = ngược

Euclidean Distance

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- Khoảng cách “đường chim bay” trong n chiều
- 0 = trùng nhau, càng lớn = càng xa
- 1536 chiều: công thức y hệt, chỉ nhiều i hơn

Lưu ý: Hầu hết vector store mặc định dùng **cosine**. Không cần tự code — nhưng cần hiểu score = 0.87 vs 0.31 nghĩa là gì.

Tính trên giấy hoặc máy tính (3 phút), post đáp án trên Discord:

Cặp 1

$$\vec{A} = [1, 2, 3] \quad \vec{B} = [2, 4, 6]$$

$$\cos(\vec{A}, \vec{B}) = ?$$

Gợi ý: $\vec{A} \cdot \vec{B} = 1 \times 2 + 2 \times 4 + 3 \times 6$

Cặp 2

$$\vec{C} = [1, 0, 0] \quad \vec{D} = [0, 1, 0]$$

$$\cos(\vec{C}, \vec{D}) = ?$$

Gợi ý: hai vector này có điểm chung nào không?

Lưu ý: Câu hỏi thêm: Cặp 1 có cosine = 1.0 mặc dù $\vec{B} = 2 \times \vec{A}$. Vì sao? Điều này nói gì về cosine similarity so với Euclidean distance?

Câu A	Câu B	Cosine	Nghĩa
“Chính sách hoàn tiền”	“Quy định đổi trả”	~ 0.87	Rất gần nghĩa
“Chính sách hoàn tiền”	“Cách giao hàng nhanh”	~ 0.52	Liên quan nhẹ
“Chính sách hoàn tiền”	“Thời tiết hôm nay”	~ 0.31	Không liên quan
“Chính sách hoàn tiền”	“Refund policy”	~ 0.82	Cross-lingual!

Embedding model hiểu nghĩa **cross-lingual**: “hoàn tiền” và “refund” gần nhau dù khác ngôn ngữ. Đây là sức mạnh so với keyword search.

Use case	Cách hoạt động	Ví dụ thực tế
Semantic search	Embed query + embed tài liệu, tìm cosine cao nhất	User hỏi “trả hàng” → tìm được chunk “chính sách đổi trả” dù không trùng từ
Clustering	Gom các vector gần nhau thành nhóm	10,000 ticket CS → tự phân thành 15 chủ đề (hoàn tiền, lỗi app, giao hàng...)
Dedup	So cosine giữa các cặp, đánh dấu cặp > threshold	Phát hiện 2 bài FAQ nội dung gần giống, gộp lại
Recommendation	Embed user intent + embed item, rank theo cosine	User vừa đọc “React hooks” → gợi ý bài “useState patterns”

Lưu ý: Embedding không tự tạo ra “sự thật”. Nếu text nguồn sai, thiếu, hoặc chunk xấu thì retrieval vẫn tệ — **garbage in, garbage out**.

Model	Dim	MTEB Avg	Khi nào dùng
text-embedding-3-small	1536	62.3	Demo, PoC, lab (rẻ, nhanh)
text-embedding-3-large	3072	64.6	Production cần chất lượng
Open-source (e5, BGE)	768–4096	61–66	Self-host, data nhạy cảm, free

- Không có model “tốt nhất” cho mọi use case. Chọn theo **quality, latency, storage, language**.
- Với product thực tế, model cân bằng thường tốt hơn model mạnh nhất nhưng chậm và đắt.

Trên thực tế có hàng trăm embedding model (Cohere, Voyage, Gemini, Jina...). Xem bảng xếp hạng đầy đủ: MTEB Leaderboard trên Hugging Face.

```
from openai import OpenAI
import numpy as np

client = OpenAI()
texts = ["Chính sách hoàn ð tin", "Quy định ð i ð tr", "ðThi ð tit hôm nay"]

resp = client.embeddings.create(
    model="text-embedding-3-small", input=texts
)
vecs = [np.array(d.embedding) for d in resp.data]

# Cosine similarity: ðnghĩa ð gn => score cao
cos = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos(vecs[0], vecs[1])) # ~0.87 hoàn ð tin <-> ð i ð tr
print(cos(vecs[0], vecs[2])) # ~0.31 hoàn ð tin <-> ðthi ð tit
```

Vector Store

Vector store không chỉ lưu vector; nó lưu cả nguồn, metadata, và khả năng lọc để retrieval có ngữ cảnh đúng

05

Mỗi record gồm 4 thành phần:

ID

Original chunk (text gốc)

Embedding vector (float array)

Metadata (key-value)

Lưu trong
vector store

Search results (top-k + scores)

Output khi query, không phải thứ được lưu

Ví dụ 1 record thật:

Trường	Giá trị
ID	policy-returns-001
Chunk	"Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem."
Vector	[0.012, -0.034, 0.091, ...] (1536 số)
Metadata	source: refund-v3.pdf category: support updated: 2026-03-01 access: public-internal

Vector dùng để *tim* (semantic search).
Chunk dùng để *inject* vào prompt cho LLM.
Hai thứ khác nhau, lưu cùng nhau.

Trường metadata	Ví dụ	Tác dụng
Source / file name	refund-policy-v3.pdf	truy vết và hiển thị nguồn
Category	finance, support, legal	lọc đúng domain trước khi search
Time / freshness	2026-03-01	tránh dùng nội dung cũ
Access level	public-internal, restricted	giới hạn truy cập
Section / chunk id	returns.section.4	debug và cite chính xác hơn

Retrieval tốt cần semantic similarity kết hợp metadata filtering

Tham số	Ý nghĩa	Ví dụ
Top-k	Lấy bao nhiêu chunk gần nhất	n_results=3: đủ cho hầu hết câu hỏi đơn. n_results=10: khi cần tổng hợp nhiều nguồn
Score threshold	Bỏ chunk có cosine thấp hơn ngưỡng	Threshold = 0.7: chỉ giữ chunk thật sự liên quan. Threshold = 0.4: chấp nhận liên quan nhẹ
Attribute filter	Lọc theo metadata trước khi search	where={"category": "support"}: chỉ tìm trong tài liệu CS, bỏ qua finance, legal

Filter theo category trước → semantic search → lấy top-3 có score > 0.6. Retrieval tốt thường đến từ **chunk đúng + metadata đủ + filter hợp lý** — không phải đổi model.

```
import chromadb

client = chromadb.Client()
col = client.create_collection("policies")

# 1. Add: chunk + metadata -> vector store
col.add(
    ids=["p1", "p2"],
    documents=["Khách hàng có 30 ngày đ▯ đ▯ i ▯ tr.",
               "Hoàn ▯ tin trong 7 ngày làm ệvic."],
    metadatas=[{"cat": "returns"}, {"cat": "refund"}],
)

# 2. Query: semantic search
results = col.query(query_texts=["đ▯ i size"], n_results=2)

# 3. Inject: dùng ▯ kt ▯ qu làm context cho LLM
context = "\n".join(results["documents"][0])
```

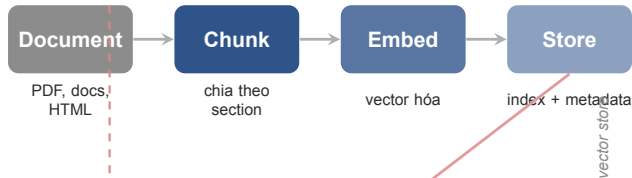
Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

06

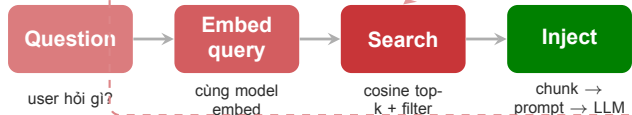
Ingestion

chạy 1 lần



Retrieval

mỗi câu hỏi



Lab 7 scope

Lưu ý: Ingestion chạy offline khi dữ liệu thay đổi. Retrieval chạy online mỗi khi user hỏi. **Hai pha này tách biệt nhau.**

1 file PDF / doc (nhiều section)

Chỉnh sách doi tra
Khách hàng có 30 ngày...
Chỉnh sách hoàn tiền
Hoàn tiền trong 7 ngày...
Giao hàng
Miễn phí đơn trên 500k...

⇓ chunking

Chunk 1: chỉnh-sách-doi-tra (150 tokens)

Chunk 2: chỉnh-sách-hoàn-tiền (120 tokens)

Chunk 3: giao-hàng (90 tokens)

Chunking = chia tài liệu dài thành các đoạn nhỏ hơn để embed và index riêng.

Vì sao phải chunk?

- Embedding model có giới hạn token input
- Một file dài embed thành 1 vector → không thể tìm *đoạn* liên quan, chỉ tìm *file* liên quan
- Chunk nhỏ → retrieval chính xác hơn, inject ít nhiễu hơn

Chiến lược chunk phổ biến:

- Theo **heading / section** (phổ biến nhất)
- Theo **số token cố định** (200–500 tokens)
- Theo **câu / paragraph**

Chunk quá to

Dính nhiều chủ đề vào cùng một vector, retrieve trùng nhưng inject rất nhiều.

> 1000 tokens

Chunk hợp lý

Một ý / một section trọn vẹn, có overlap 10–15% giữa các chunk liền kề.

200–500 tokens

Chunk quá nhỏ

Mất ngữ cảnh, retrieve nhiều mảnh rời rạc, khó tổng hợp thành câu trả lời tốt.

< 50 tokens

Khi cắt chunk, lấy thêm 1–2 câu cuối của chunk trước làm phần đầu chunk sau. Giúp giữ ngữ cảnh tại điểm cắt, tránh mất nghĩa giữa chừng.

Rule of thumb: bắt đầu đơn giản với chunk theo section / heading, rồi tối ưu sau bằng eval thay vì đoán cảm tính.

Tài liệu thật từ bộ phận CS — chưa qua xử lý:

=== CHÍNH SÁCH ĐỔI TRẢ & HOÀN TIỀN (v3.1 - 01/2026) ===

Liên hệ: hotline 1900-xxxx | email: cs@shop.vn

1. ĐỔI TRẢ : Khách có 30 ngày kể từ ngày nhận hàng.

Sản phẩm phải còn nguyên tem. Không áp dụng cho

đồ sale > 50%.

2. HOÀN TIỀN: 7 ngày làm việc qua chuyển khoản hoặc ví.

Lưu ý: đơn COD hoàn tiền chậm hơn (10-14 ngày).

3. GIAO HÀNG: Miễn phí đơn > 500k. Nội thành 1-2 ngày,

tỉnh 3-5 ngày. Không hỗ trợ ship quốc tế.

--- Cập nhật bởi: Nguyễn Văn A, team CS ---

--- Footer: Tài liệu nội bộ, không chia sẻ ngoài công ty ---

Nhiệm vụ (10 phút):

1. **Clean:** Bạn sẽ bỏ gì, giữ gì trước khi chunk? (header, footer, hotline?)
2. **Chunk:** Chia tài liệu trên thành các chunk. Viết ra nội dung từng chunk.
3. **Metadata:** Mỗi chunk cần gắn metadata gì? (source, category, date, ...)
4. **Predict failure:** Viết 1 câu query mà strategy của bạn sẽ trả lời **sai** hoặc **thiếu**.

Trình bày: Mỗi nhóm chia sẻ strategy + predicted failure query (2 phút/nhóm).

Lưu ý: Câu hỏi thử thách: Nếu user hỏi “đổi size áo sale 60% được không?” — chunk nào sẽ được retrieve, và LLM sẽ trả lời đúng hay sai?

Chunk quá to

Q1: Hoàn tiền mất bao lâu?

A1: 7 ngày.

Q2: Đổi size được không?

A2: Trong 30 ngày.

Q3: Ship quốc tế?

A3: Không hỗ trợ.

Query “đổi size” → retrieve cả 3 Q&A → LLM bị nhiễu bởi Q1, Q3.

Chunk hợp lý

Q2: Đổi size được không?

A2: Được, trong vòng 30 ngày kể từ ngày mua.

Query “đổi size” → retrieve đúng Q2 → LLM trả lời chính xác.

Chunk quá nhỏ

“Trong 30 ngày.”

Query “đổi size” → retrieve “Trong 30 ngày” → thiếu ngữ cảnh, LLM không biết 30 ngày cho việc gì.

Retrieval

- Tìm context liên quan cho câu hỏi hiện tại
- Đọc từ knowledge base
- Trọng tâm: **relevance** và **grounding**

Ví dụ: User hỏi “chính sách đổi trả” → agent tìm trong FAQ → trả lời dựa trên tài liệu.

Memory

- Lưu trạng thái, sở thích, lịch sử chọn lọc
- Từ user profile hoặc interaction history
- Trọng tâm: **continuity** và **personalization**

Ví dụ: Cùng user hỏi lần 2 → agent nhớ lần trước đã đổi size M → gợi ý “bạn muốn đổi lại size khác?”

Lưu ý: Nhiều hệ thống dùng cả hai: **retrieval** để biết domain facts, **memory** để biết user này là ai và đã làm gì trước đó.

Query: “Chính sách đổi trả áp dụng trong bao lâu?”

Chunk xấu (raw, không section)

Retrieved (cosine: 0.61): “...giao hàng miễn phí đơn trên 500k. Đổi trả trong 30 ngày. Liên hệ hotline 1900...”

LLM answer: “Bạn có thể đổi trả và liên hệ hotline 1900...” **(nhiều, thiếu chi tiết)**

Chunk tốt (theo section + metadata)

Retrieved (cosine: 0.89): “Chính sách đổi trả: Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem.”

LLM answer: “30 ngày kể từ ngày nhận, sản phẩm còn nguyên tem.” **(chính xác, có nguồn)**

Lưu ý: Cùng model, cùng query — khác nhau hoàn toàn vì **chất lượng chunk**.

Mỗi nhóm nghiên cứu **1 phương pháp chunking**, hiểu cách hoạt động, ưu/nhược điểm, và khi nào nên dùng. Sau đó trình bày cho cả lớp.

Phân công — mỗi nhóm chọn 1 chủ đề:

1. Recursive Character Splitting

LangChain-style, separator hierarchy

2. Semantic Chunking

Dùng embedding để tìm điểm cắt tự nhiên

3. Document-structure Chunking

Theo heading, table, list — Markdown/HTML aware

4. Agentic Chunking

Dùng LLM để quyết định chunk boundary

5. Parent-Child / Small-to-Big

Retrieve chunk nhỏ, inject chunk cha lớn hơn

6. Sliding Window + Overlap

Fixed-size với stride, trade-off coverage vs cost

Phase 1 — Nghiên cứu (20 phút):

1. Tìm hiểu phương pháp: đọc docs, blog, paper, hoặc source code
2. Chuẩn bị trình bày với 4 câu hỏi bắt buộc:

Q1. Phương pháp này hoạt động **như thế nào?** (giải thích cho người chưa biết)

Q2. **Khi nào nên dùng**, khi nào **không nên**? (ví dụ cụ thể)

Phase 2 — Trình bày (10 phút): Các bạn xung phong chia sẻ kết quả, mỗi bạn trình bày **1 phương pháp chunking chưa được trình bày** (2 phút/bạn).

Tài liệu gợi ý: LangChain docs (text splitters), LlamaIndex docs (node parsers), blog “Chunking Strategies for LLM Applications” (Pinecone).

Hands-on 7

Mục tiêu của lab là nối được dữ liệu riêng vào hệ thống AI theo pipeline tối thiểu nhưng đúng bản chất

07

Phase 1 — Cá nhân

Hoàn thành TODO trong 3 file:

1. `src/chunking.py` — 3 chunking strategies + cosine similarity
2. `src/store.py` — `EmbeddingStore` (5 methods)
3. `src/agent.py` — `KnowledgeBaseAgent` (RAG)

Document và `FixedSizeChunker` đã implement sẵn làm ví dụ. Verify: `pytest tests/ -v`

Phase 2 — Nhóm: So sánh Strategy

1. Nhóm chọn domain + thu thập 5–10 docs
2. Thống nhất 5 benchmark queries + gold answers
3. Mỗi người thử strategy riêng (chunking, metadata)
4. Chạy cùng queries, so sánh kết quả
5. Demo + thảo luận liên nhóm

Cùng code, cùng model — khác **data strategy** sẽ cho kết quả rất khác. Data quality > model selection.

Hosted / managed

File Search / managed retrieval giúp đi nhanh, giảm code hạ tầng, phù hợp demo và PoC.

Self-managed

Dùng Chroma / Faiss / vector DB khi cần kiểm soát chunking, metadata, pipeline, hoặc cost path chi tiết hơn.

Hôm nay dùng **Chroma** (self-managed) để hiểu pipeline end-to-end. Hosted option tìm hiểu thêm ngoài giờ.

Nộp bài

- src/ — hoàn thành tất cả TODO
- report/REPORT.md — 1 báo cáo / sinh viên

Phần	Điểm
Cá nhân (code + phân tích)	60
Nhóm (strategy + so sánh)	40
Tổng	100

5 góc nhìn tự đánh giá

1. **Retrieval Precision** — top-3 có đúng không?
2. **Chunk Coherence** — chunk giữ được ý trọn vẹn?
3. **Metadata Utility** — filter có giúp tăng chính xác?
4. **Grounding Quality** — answer dựa trên context?
5. **Data Strategy Impact** — strategy phù hợp domain?

- ✓ **Tests pass:** `pytest tests/ -v` không có FAIL
- ✓ **3 chunking strategies** đều implement và so sánh được
- ✓ **EmbeddingStore** search + filter + delete hoạt động
- ✓ **KnowledgeBaseAgent** trả lời dựa trên retrieved context
- ✓ **5 benchmark queries** có gold answers và kết quả so sánh
- ✓ **Report** giải thích approach + failure cases

Key Takeaways

Day 07 đặt nền cho RAG và mọi hệ thống AI dùng tri thức riêng

08

1. **Data quality** thường quan trọng hơn đổi sang model đắt hơn.
2. **Embedding** là lớp dịch ngôn ngữ sang không gian có thể so sánh nghĩa.
3. **Vector store** là bộ nhớ dài hạn có thể tìm kiếm bằng ngữ nghĩa và metadata.
4. **Retrieval pipeline** là cầu nối từ dữ liệu riêng tới câu trả lời grounded của agent.

- Stanford HAI. *AI Index 2025*. <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- OpenAI. *Embeddings Guide*. <https://platform.openai.com/docs/guides/embeddings>
- OpenAI. *File Search*. <https://platform.openai.com/docs/guides/tools-file-search>
- Chroma Docs. <https://docs.trychroma.com/>

- HuggingFace. *MTEB Benchmark*. <https://huggingface.co/blog/mteb>
- Karpukhin et al. *Dense Passage Retrieval*. arXiv:2004.04906

Danh sách đầy đủ có trong lab handout.

Bài tập sau buổi

- Hoàn thiện solution.py nếu chưa pass hết tests
- Rà lại knowledge base: bỏ nội dung nhiễu, thêm metadata
- Thử thêm 3 queries khó hơn, tìm failure cases mới

Preview Day 08

Ngày 8 đi tiếp sang **RAG pipeline**: indexing, retrieval, generation, evaluation.